Brownout Survival

Eli Keetch

January 2026

1 Overview

Embedded System That Survives Brownouts Without Resetting or Corrupting Data

This project focuses on designing an embedded system that can tolerate power instability by detecting brownouts early, preserving critical system state, entering a safe shutdown, and restoring operation predictably when power returns.

1.1 One-Sentence Summary

The purpose of this project is to design an embedded system that detects power instability, saves critical state quickly, shuts down safely, and restores operation correctly when power returns.

2 Problem and Motivation

Most microcontroller projects assume stable power. In real systems, voltage dips occur due to weak power adapters, motors starting, loose cables, or users unplugging devices unexpectedly. During a brownout, a microcontroller may continue executing instructions incorrectly before fully resetting, leading to unpredictable behavior or corrupted memory.

This project demonstrates how to design embedded systems that function reliably under real-world power conditions, in which can be unreliable depending on the system such as sudden power draws like starting a large motor or things like that.

3 Objectives

The system must be able to:

- Detect a brownout early, before the MCU becomes unreliable
- Preserve only critical system state, not full RAM

- Prevent corruption during state saves using checksums or valid markers
- Enter a safe shutdown state with outputs driven to known values
- Recover predictably on restart by restoring valid state or reverting safely

4 System Overview

The system consists of three major functional blocks.

4.1 Power Awareness

The microcontroller monitors its supply voltage using an ADC connected to a voltage divider (or a comparator if available). When voltage drops below a predefined warning threshold, the firmware initiates a save-and-shutdown sequence.

4.2 State Preservation

A small snapshot of critical system state is written to nonvolatile memory such as EEPROM, flash, or FRAM. Each snapshot includes:

- A version identifier
- A sequence number
- The critical state payload
- A CRC or checksum
- A valid or commit flag (or a double-buffering scheme)

  This allows the system to detect incomplete or corrupted writes.

4.3 Recovery Logic

On startup, the firmware checks whether the previous shutdown was clean. If a valid state snapshot is detected, it is restored and operation continues. If the snapshot is invalid, the system safely falls back to default values.

5 Hardware Components

Planned hardware components include:

- Microcontroller board (Arduino, ESP32, or similar)
- Voltage divider and ADC input (or comparator / brownout IC)
- Bulk capacitor to provide short hold-up time during power loss

- Nonvolatile memory (on-board EEPROM/flash or external FRAM)
- Output load for demonstration:
  - LEDs (simple)
  - Relay or motor driver (more advanced)

6 Firmware Components

Planned firmware features include:

- Continuous voltage monitoring loop or interrupt
- Configurable brownout warning and shutdown thresholds
- State machine handling normal operation, saving, and shutdown
- Structured state packet with integrity checking

7 Testing Plan

To verify correct operation, the following tests will be performed:

- **Hold-up measurement**: measure how long the MCU continues running after power is removed using the bulk capacitor
- **Brownout demonstration**: repeatedly dip supply voltage and show correct recovery behavior
- **Corruption test**: interrupt power mid-write and verify invalid state is detected
- **Cycle test**: perform 25–50 brownout cycles and record success rate

8 Deliverables

The final project submission will include:

- Block diagram and wiring diagram
- Fully commented firmware code
- Short written report describing:
  - Measured hold-up time
  - Threshold selection
  - State format and integrity method
- Live demonstration of brownout survival

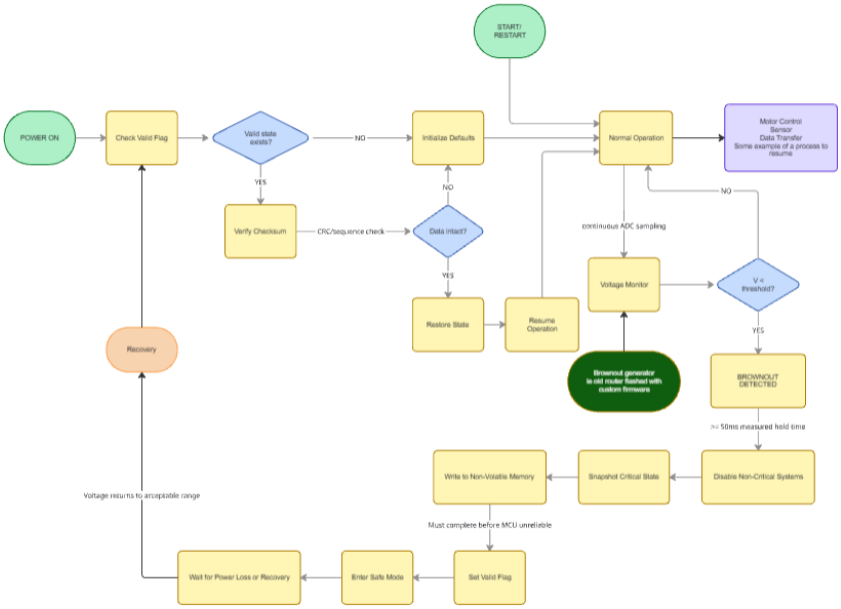


Figure 1: Firmware logic flow for brownout detection, state saving, and recovery.

9 Stretch Goals

If time allows, optional enhancements include:

- External brownout detector IC or comparator interrupt
- Double-buffered storage for improved corruption resistance
- Serial logging of brownout events

10 Timeline

The project will be developed incrementally across the academic semester to allow time for design iteration, testing, documentation, and integration with other coursework.

10.1 Weeks 1–2: Project Definition and System Design

- Finalize project requirements and success criteria
- Choose MCU platform and nonvolatile storage method (EEPROM/flash/FRAM)
- Create block diagram and firmware flow diagram (Figure 1)

- Define what "critical state" must be preserved for recovery

10.2 Weeks 3–4: Power Monitoring Circuit and Validation

- Build voltage sensing circuit (ADC divider or comparator input)
- Implement voltage measurement firmware and validate against a multimeter
- Establish initial warning and critical thresholds with safety margin
- Verify stable detection behavior (noise filtering / averaging as needed)

10.3 Weeks 5–6: State Packet Format and Nonvolatile Storage

- Design state packet structure (version, sequence number, payload, CRC, commit flag)
- Implement read/write routines for nonvolatile memory
- Implement integrity validation (CRC/checksum + valid marker)
- Confirm the system rejects invalid or incomplete state packets

10.4 Weeks 7–8: Brownout Handling State Machine and Safe Shutdown

- Implement the firmware state machine (normal $\rightarrow$ detect $\rightarrow$ save $\rightarrow$ safe shutdown)
- Set outputs to known safe values during shutdown
- Verify state save occurs within the allowable time window after detection
- Perform early brownout testing with controlled power dips

10.5 Weeks 9–10: Hold-Up Time Hardware and Timing Characterization

- Add bulk capacitance for hold-up time during power loss
- Measure voltage decay and available runtime after power removal
- Measure save time and ensure it fits within the hold-up budget
- Adjust thresholds and firmware timing based on measured results

10.6 Weeks 11–12: Demonstration Tests and Reliability Cycling

- Run repeated brownout cycles (25–50) and record success/failure outcomes
- Perform mid-write power interruption tests to verify corruption immunity
- Verify deterministic recovery: restore last valid state or revert safely
- Finalize the demonstration procedure and success criteria

10.7 Weeks 13–14: Documentation and Final Integration

- Finalize wiring diagram, block diagram, and firmware documentation
- Compile measurement data (hold-up time, thresholds, save timing)
- Prepare short written report and demonstration materials
- Perform full end-to-end rehearsal of the live demo

10.8 Week 15: Buffer Week and Stretch Goals

- Address last-minute bugs discovered during final testing
- Implement stretch goals if time allows (double-buffering, logging, comparator interrupt)
- Final cleanup and submission

11 Demonstration of Effectiveness

The effectiveness of the proposed system will be demonstrated through a structured comparison between a baseline system without brownout handling and the completed brownout-resilient design. The demonstration is designed to clearly show failure modes under power instability and to verify correct behavior, data integrity, and recovery when brownout handling is enabled.

11.1 Baseline Failure Demonstration

To establish a control case, the system will first be operated with brownout detection and state preservation disabled. The microcontroller will execute a simple task involving a visible or logged state, such as an operating mode number, LED brightness level, or counter value.

During operation, the supply voltage will be intentionally interrupted or dipped. In this configuration, the system is expected to exhibit unpredictable behavior, including state loss, output glitches, or inconsistent restarts. This baseline demonstration highlights the real-world problem that the project is designed to solve.

11.2 Brownout Survival Demonstration

With brownout handling enabled, the same operating task will be repeated. The system will monitor its supply voltage and detect a brownout condition before the microcontroller becomes unreliable.

When a brownout is detected, the firmware will:

- Save critical system state to nonvolatile memory
- Verify write integrity using a checksum or CRC
- Drive outputs to predefined safe values
- Enter a controlled shutdown state

After power is restored, the system will automatically validate the stored state and resume operation from the last known good state. The demonstration will show that the system continues seamlessly, without state loss or corruption, despite repeated power interruptions.

11.3 Corruption Immunity Demonstration

To demonstrate robustness against incomplete writes, power will be intentionally removed during the state save process. Upon restart, the firmware will detect the invalid or partially written state using integrity markers and reject it.

In this case, the system will safely revert to default values rather than attempting to restore corrupted data. This demonstration confirms that the system not only preserves state, but does so in a manner that prevents undefined or unsafe behavior.

11.4 Quantitative Hold-Up Time Measurement

The available hold-up time provided by the bulk capacitor will be measured by recording the duration between power removal and microcontroller shutdown. Voltage decay and firmware timing measurements will be used to verify that sufficient time exists to complete state preservation and safe shutdown operations.

These measurements provide quantitative validation that the hardware design supports the required firmware response time.

11.5 Repeatability and Reliability Testing

To demonstrate reliability, the system will undergo repeated brownout cycles (approximately 25–50 iterations). Each cycle will log whether state restoration was successful or safely rejected.

The demonstration will show a high success rate with no acceptance of corrupted state, reinforcing that correct operation is repeatable and not dependent on timing luck.

11.6 Demonstration Summary

The demonstration results will be summarized by comparing system behavior with and without brownout handling. The final system is expected to show deterministic behavior, safe recovery, and data integrity across all tested brownout scenarios, validating the effectiveness of the design under realistic power conditions.